Metanoia — Technical Walkthrough (Version 2)

Autonomous, mandate-bound procurement for API/software subscriptions, settled through Juspay Hyperswitch — deployed on Google Cloud

Aayush Shrestha

2026-07-23 · code snapshot commit 4dfc45b

Contents

How to read this guide. This is a detailed learning companion, written to be understood when read aloud (for a NotebookLM audio overview). Every section starts in plain English, then adds the technical detail. Diagrams are drawn in text and then explained in words, so nothing depends on seeing a picture. No credentials, secrets, database passwords, or full API keys appear anywhere in this document.

Every claim is tagged one of three ways: **[PROVEN]** — observed working against live infrastructure; **[CODED-UNTESTED]** — implemented and type/logic-checked, but not yet observed end to end; **[DEFERRED]** — intentionally not built yet.

This is the longer learning guide, not the three-page interview submission. Where Version 1 described a local prototype, Version 2 describes the same system now **deployed on Google Cloud**, with several bugs fixed, a payment test lab, per-browser isolation, refunds, and an honest, fully diagnosed webhook story.

---

1. What Metanoia is, and the vertical it targets

Metanoia lets you hand an AI agent a **budget instead of your credit card**.

You tell it what capability you need — for example, "I need a transcription API for about ten dollars a month" — and you set a standing **spending mandate**. In the demo that mandate is: at most sixty dollars a month in total, at most forty dollars per single charge, and at most a few active subscriptions at once. The agent then researches a marketplace, compares the best-fitting offers, ranks them with a fixed formula, checks the winner against your mandate, asks you to confirm, and only then settles a real sandbox payment through Juspay Hyperswitch. After paying, it proves the thing it bought actually works by making an authenticated call to the purchased service.

The one sentence that captures the whole design is: **the model proposes, and trusted server code decides.** A language model can research, compare, and recommend, but it can never set a price, choose the final plan, or move money. A deterministic ranker and a deterministic spending gate are the only authority, and they run on data the server owns, not on numbers the model made up.

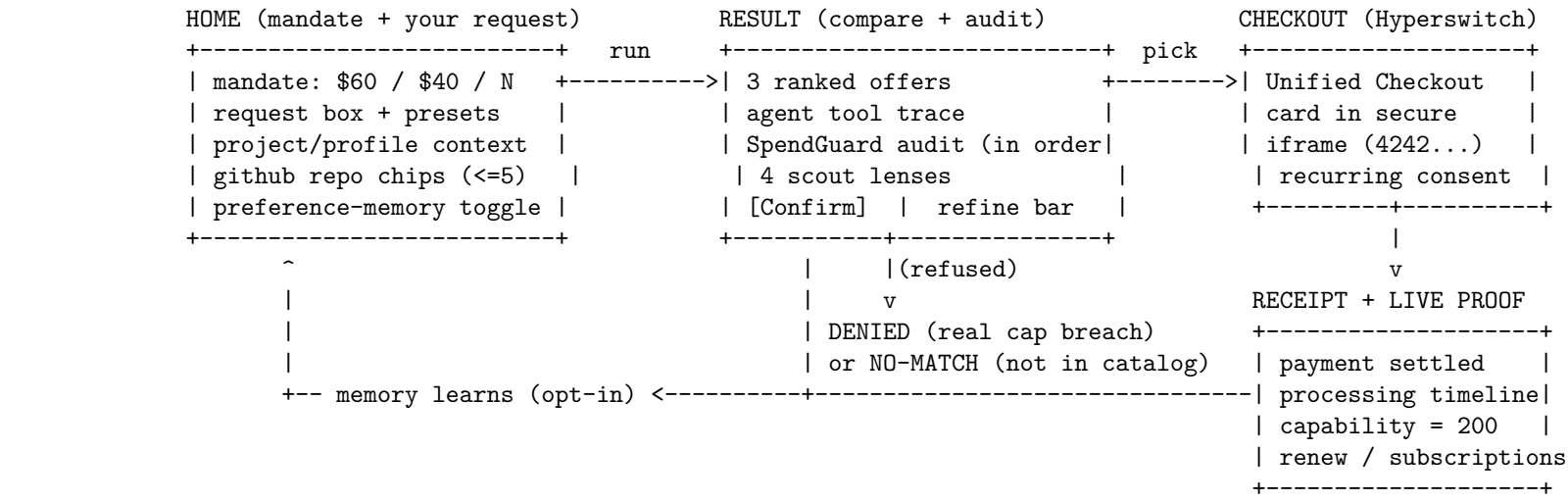
The vertical is developer and API/software subscriptions, chosen on purpose. These "products" are recurring, priced, comparable, and machine-describable — they have a price, a throughput number, an uptime number, and feature flags — which is exactly what a deterministic scoring formula needs. Developers already feel subscription sprawl, so the pain is real. And it avoids physical shipping, which keeps the demo honest about what a payment actually proves. The name *metanoia* is Greek for a fundamental change of mind: the thesis is a change in how software gets bought, from a human filling forms to an agent operating under a machine-checkable mandate.

---

2. The complete user journey, from request to sandbox payment

Plain English: you land on a workbench, describe what you need, run the agent, compare three ranked options with a live audit of every rule it checked, confirm one, pay with a test card inside Hyperswitch's secure form, and land on a receipt that proves the capability works.

Here is the journey as a text diagram. Read it left to right, top to bottom.



In words: the **Home** screen shows your standing mandate as three cards (monthly cap, per-charge cap, and how many subscriptions you allow), plus a box to type your request and optional context about you and your project. You can add up to five public GitHub repository links, which the server reads as background data only. When you press Run, the browser calls one server route, `POST /api/agent/plan`, and an honest "processing" screen appears — it does not fake per-step progress; it just says the pipeline is running.

The **Result** screen shows three ranked offers in a comparison table, the agent's tool trace, the spending audit with each rule shown in the order it was checked, and four independent "scout" opinions. There is also a refine bar: if the recommendation is not quite right, you can add feedback chips or free text and re-run, which is the "structured refinement" loop.

There are two honest refusal shapes. If real offers exist but every one breaks your mandate, you get a red **Denied** screen that names the exact failing rule and says the card was never touched. If your request is simply not something the catalog carries, you get a neutral **No-match** screen that explicitly says the budget was not the problem — there was just nothing to compare. Keeping these two apart is a deliberate honesty decision.

When you confirm, the browser goes to **Checkout**, which runs the spending gate again on the server, then mounts Hyperswitch's Unified Checkout. You type the test card into Hyperswitch's own secure iframe; the card never touches Metanoia's server. On success you land on the **Receipt**, which verifies the payment authoritatively, records the subscription, issues a scoped credential, and then makes a real authenticated call to the purchased capability and shows the response. If you opted into memory, the run is remembered so the next one is personalized. [**PROVEN**] end to end, both locally and on the live Cloud Run deployment (a real in-browser checkout was completed against the deployed app).

---

3. The decision core: Gemini agent, four scouts, deterministic ranking, structured refinements, SpendGuard

This is the heart of the system. Five things cooperate, and only two of them have authority.

3.1 The one Gemini agent that proposes

Plain English: a single Gemini model runs a short tool loop. It can look at the catalog, ask whether a plan is allowed, and submit exactly one structured proposal. It cannot call any payment function — there is no payment tool within its reach.

The model is `gemini-3.1-pro-preview` on Vertex AI, driven through the AI SDK's tool-loop agent. Its only three tools are: `list_services`, which returns the marketplace with comparable attributes; `check_mandate`, which gives an authoritative yes/no from the spending gate using server-side prices; and `recommend`, which submits the final structured proposal exactly once, validated by a strict schema. The loop stops when the model calls `recommend` or after a bounded number of steps. A unit test asserts that the agent's source file never even imports the payment functions, so "the model cannot spend" is enforced by code, not by trust. [**PROVEN**]

3.2 Deterministic ranking (the math, not the model's opinion)

Plain English: the AI does not score the options. A fixed formula does, on numbers the server owns, so the ranking is reproducible and explainable.

For each plan in the requested capability, the formula computes four sub-scores between zero and one: feature coverage (how many required features it has), price efficiency (cheaper relative to your ceiling scores higher), reliability (mapped from uptime), and throughput (its requests-per-second versus your target). These are combined with weights that depend on your stated priority — cost, balanced, reliability, or throughput. For example, a cost priority weights price most heavily, while a throughput priority weights requests-per-second most heavily. The result is a score from zero to a hundred.

Separately, there are **hard constraints** that make a plan ineligible no matter its score: a price above an explicit maximum, a throughput below an explicit minimum, or a missing required feature. The final sort is: eligible plans first, then by higher score, then by lower price. A plan is eligible only if it has no hard failures **and** the spending gate approves it. This is why a very high-quality but too-expensive plan is shown as "blocked," not recommended. [**PROVEN**] (a test shows the model selecting a premium plan and the server choosing the compliant one instead).

3.3 The four parallel scouts (advisory only)

Plain English: after the shortlist exists, four independent analyst agents review it at the same time, each from a different angle. They give opinions and evidence; they never decide or pay.

The four lenses are **Price** (cheapest among equally qualified), **Value** (feature coverage per dollar), **Quality** (uptime, throughput, operational fit), and **Market Signal** (a grounded scan of the real external market). The first three read only the internal catalog. The fourth uses Vertex's Google Search grounding to name a few real external products as research-only context, clearly labeled so a viewer never mistakes them for something purchasable here. The scouts run concurrently; if any one fails, it comes back as "unavailable" and never blocks the purchase. Their catalog picks are sanitized so a scout cannot smuggle in a plan the ranker never approved. The panel returns four reports. [**PROVEN**] for the panel returning four lenses; the *content quality* of the external grounded result depends on Vertex Google Search at run time, so treat specific external findings as [**CODED-UNTESTED**].

3.4 SpendGuard, the deterministic spending gate

Plain English: SpendGuard is a pure function that decides whether a proposed purchase is allowed, and it produces an ordered list of every rule it checked.

It evaluates, in order: is the mandate still valid (not expired); is the single charge within the per-charge cap; does the running monthly total plus this item stay within the monthly cap; category and merchant allowlists if configured; and the active-subscription count. The purchase is approved only if every check passes. The verdict includes how much budget would remain and a human-readable summary, and the UI renders these

exact checks as the on-screen audit. Over-budget requests are refused before any charge, and the checkout route returns an HTTP 403 without ever contacting Hyperswitch. [**PROVEN**], and covered by unit tests.

3.5 Structured refinements

Plain English: if the recommendation is not what you wanted, you refine it with structured, server-enforced feedback rather than starting over. The result screen has a refine bar with quick modes ("cheaper", "higher throughput", "more reliable", "different vendor") plus free text. Each mode is converted on the server into a concrete, deterministic constraint over the catalog — for example "cheaper" caps the max price below the current pick and excludes it; "different vendor" excludes every plan from that vendor — via `applyProcurementRefinement` in `lib/agent/refinement.ts`. The model still interprets any free-text feedback, but it cannot ignore the enforced constraint: the deterministic ranker re-runs with those exclusions and the mandate is re-checked. The authority never changes — refinement steers the proposal, not the decision. [**PROVEN**] (unit-tested in `refinement.test.ts` and for the new categories).

---

4. Why the model proposes but trusted server code decides

Plain English: the single most important safety property is that the language model never has the final say over money. It is a research-and-suggestion engine; a small amount of deterministic, testable server code is the authority.

Concretely, four independent barriers enforce this:

1. **No payment tool in reach.** The agent's toolset is list, check, and recommend. It cannot call the function that creates a payment or the function that charges a saved card. A test asserts the agent module does not import those functions at all.
2. **Server-owned prices.** The server's `decide()` step recomputes the amount and the spending verdict from the catalog. The number the model wrote in its proposal is never used to authorize anything.
3. **The server overrides the model's pick.** `decide()` ignores the model's chosen plan id, runs the deterministic ranker, and selects the highest-ranked eligible plan. If the model proposed something over-cap, the server picks the compliant one instead, and a test proves this override.
4. **A second gate at checkout.** The checkout route runs SpendGuard again before creating any Hyperswitch intent, and renewals run it a third time before any off-session charge.

The design borrows the "grounding gate" idea: the model can be creative, but only verified, server-computed facts get through to an action. This is what makes the system safe to describe as "an agent with a card that cannot overspend." [**PROVEN**]

---

5. Payments: Hyperswitch checkout, CIT, idempotency, Fauxpay, consent, credentials, refunds, cancellations, resubscriptions

Plain English: payments go through Juspay Hyperswitch, an orchestrator that sits in front of many payment processors. The browser collects the card inside Hyperswitch's own secure form; Metanoia's server never sees raw card numbers. Payment IDs are deterministic so retries do not double-charge. When a payment succeeds, the server issues a scoped credential for the purchased capability.

5.1 Unified Checkout and the customer-initiated transaction (CIT)

The first payment is a **customer-initiated transaction**: you are present and you confirm it. Metanoia's server creates a payment intent through Hyperswitch with "confirm later" set, so the browser SDK is what actually confirms with the card. The card data lives only inside Hyperswitch's hosted iframe; the merchant

server receives only tokens. This is the PCI boundary, and it is preserved. [**PROVEN**] — a real checkout completes and settles.

5.2 Fauxpay, and the honest recurring limitation

Fauxpay is Hyperswitch's dummy sandbox connector. It reliably succeeds and is used for the checkout, so it proves the sandbox settlement path end to end. But Fauxpay **does not return a reusable saved payment method**, which means it cannot demonstrate real recurring billing. Be exact about this: **Fauxpay proves sandbox checkout, but it does not prove saved-card recurring (merchant-initiated) billing.** [**PROVEN**] for checkout; recurring is **not** proven.

5.3 Stable payment IDs and idempotency (and a bug this fixed)

Payment IDs are deterministic. A stable ID is derived from a seed so that retrying the same checkout attempt reuses the same ID and never creates a second charge. In Version 1 the seed was "customer, plan, and billing period." Version 2 corrected a real bug in that scheme (see the bugs section): the ID now reuses only within a single checkout attempt, so a fresh subscribe after a cancellation gets a new ID and charges again, while honest double-clicks of the same attempt stay idempotent. Confirming the same attempt twice records exactly one subscription. [**PROVEN**] by tests.

5.4 Recurring consent

Plain English: because a subscription implies future charges, the checkout now asks for explicit consent.

The checkout form shows Hyperswitch's own "save this payment method for future use" control and a plain disclosure that subscribing authorizes a monthly charge until you cancel, and the Pay button is gated until you acknowledge it. At the protocol level, when the Unified Checkout SDK is used, Hyperswitch's saved-method control is what captures the customer's acceptance. This makes the consent affordance real and visible. [**CODED-UNTESTED**] as a full recurring setup, because a reusable method cannot actually be banked on Fauxpay.

5.5 Credential issuance and the capability proof

When a payment is verified as succeeded, the server issues a deterministic, scoped credential for that customer and plan. The receipt then makes a real authenticated call to the purchased capability's internal endpoint, which returns the data only if the credential is valid and matches the plan, and otherwise returns 401. This is the "buy it, then immediately use it" loop, and it is genuinely end to end — but it is important to be honest that the provider is an **internal authenticated sandbox mock**, not a real outside company. [**PROVEN**] — a valid credential returns 200; a missing or wrong one returns 401.

5.6 Refunds, cancellations, and resubscriptions

Plain English: you can refund a payment, cancel a subscription, and resubscribe, and each of these behaves correctly and safely.

- **Refunds** are server-side only and go through four guardrails: the payment must be **owned by your browser session**; it must be **succeeded** (checked by retrieving the live payment); it is **idempotent** (a deterministic refund ID means clicking twice resolves to the same refund); and the returned status is **authoritative**, taken from retrieving the refund after creating it, not from the create response. The verified refund is then persisted so it survives a page reload. One real refund was executed against the sandbox and came back "succeeded." [**PROVEN**] for a single real refund and for the guardrail logic (which has unit tests).
- **Cancellation** frees the monthly budget and revokes the capability credential immediately, so access actually stops. For a real off-session mandate this is also where Hyperswitch's mandate-revoke call would go, which is wired conceptually but tied to the recurring path. [**PROVEN**] for the cancel-and-free-budget behavior.

- **Resubscribing after a cancel** creates a fresh payment ID and charges again, which is the corrected behavior from the bug fix in section 5.3. [**PROVEN**] by a dedicated test.

---

6. The Payment Test Lab: success, decline, insufficient funds, 3DS, and refund

Plain English: there is a dedicated page that lets you exercise real payment outcomes safely, using Hyperswitch's official dummy-connector test cards. It never auto-fires payments and never types cards into the secure iframe for you.

The lab shows scenario cards, each with a card number and a **Copy** button. You copy a card, open a real checkout yourself, and paste it into Hyperswitch's secure form — the lab deliberately does **not** inject cards into the iframe. The dummy connector decides the outcome from the card number, so you get deterministic scenarios:

- **Success** — Visa, Mastercard, and Amex test numbers, all settle.
- **Decline** — a generic hard-decline card.
- **Insufficient funds** — a card that fails for that specific reason.
- **3DS challenge** — a card that returns "requires customer action" with a redirect step.

Below the scenarios, the lab lists the payments **your session** created. Each row shows the payment ID, connector, status, and a small timeline, and has a **Refresh status** button that fetches the live status one row at a time (so a page load never fans out into many calls, which matters for sandbox rate limits). A succeeded row has a **Refund** button that runs the server-side, idempotent, session-owned refund and shows the verified result. Error handling is honest: a connector shows "unknown" until you refresh, and a failed lookup shows "unavailable" rather than guessing a cause. PayPal is shown as "pending credentials" with no fake button, because adding a non-functional wallet button would be dishonest. [**PROVEN**] for the lab, the copy flow, live status refresh, and one real refund; the decline and 3DS card outcomes are [**CODED-UNTESTED**] in the sense that the scenario cards are wired but not each individually screen-captured.

---

7. Session isolation and protection against cross-user payment access

Plain English: because the deployed app is public, every visitor must see and act on only their own payments — never anyone else's. Version 2 makes each browser its own isolated tenant.

The mechanism is a **per-browser identity**: an opaque, unguessable, high-entropy value stored in a cookie that JavaScript cannot read (http-only) and that is only sent over HTTPS in production (secure). That value is the ownership token itself. There is deliberately **no shared signing secret** involved, so there is no weak development fallback secret to worry about. This per-session identity replaced the old single shared demo customer everywhere — payments, subscriptions, credentials, the lab, and preference memory are all now scoped to the session.

Two consequences matter for safety. First, **payment IDs are seeded with the session identity**, so two different browsers buying the same plan get *distinct* payment IDs; that closes a real cross-session hole, because otherwise the IDs would have collided. Second, **ownership is a simple, exact match**: a refund or a status lookup is allowed only if the payment's owner equals the current session. A different session, an anonymous session, or a tampered/truncated identity all own nothing and are refused. This is covered by unit tests: the owner is allowed, a different session is blocked, an anonymous session is blocked, a tampered/wrong identity is blocked, and a nonexistent payment owns nothing. [**PROVEN**]

---

8. Data: Drizzle, Cloud SQL, payment records, webhook events, preference memory — and their separation

Plain English: locally everything runs in memory (persisted to a JSON file so pages share state). In production it runs on Cloud SQL Postgres. Payment data and preference-memory data live in **separate schemas** and never reference each other.

The storage layer is defined by two interfaces — one for payments, one for preference memory — and each has two interchangeable backends: an in-memory implementation (disk-backed for local development) and a Postgres implementation. The app selects Postgres automatically when the Cloud SQL settings are present. The Postgres connection uses Google's Cloud SQL connector, which opens an authenticated socket to the instance by its connection name, using the same Google identity the rest of the app uses, so there is no public IP to manage. **Drizzle** is the typed schema-and-query layer on top.

The **payment schema** holds: `attempts` (one row per payment ID, which is how idempotent recording works), `subscriptions` (keyed by customer and plan, so a renewal updates in place), `credentials` (issued at most once per customer and plan), `events` (keyed by event ID, so webhook de-duplication is a database constraint, with the raw payload retained), and — new in Version 2 — `refunds` (the latest verified refund per payment, so the lab can show it after a reload). The `events` table also gained two columns so the reconciliation sweep can settle retained events with full fidelity.

The **preference-memory schema** is separate: consent, extracted facts, choice events, and source references. There is no foreign key crossing from memory into payments. Preference memory is off by default, only writes when consent is granted, stores derived facts (never raw social data or tokens), personalizes the next run's ranking priority, and is fully deletable. [**PROVEN**] locally for all of the above; [**PROVEN**] on Cloud SQL for the payment path (the deployed app's database-backed pages work and migrations were applied).

8.1 Webhook events and reconciliation

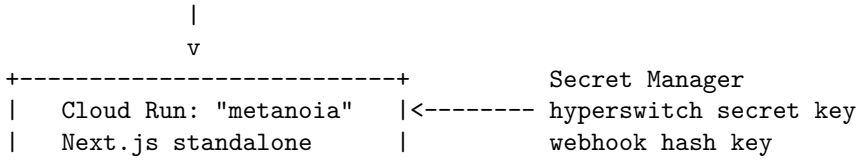
The webhook receiver reads the **raw request body** and verifies an HMAC signature over it before parsing — SHA-512 or SHA-256, compared in constant time. An invalid or missing signature returns 401. A valid one is processed in a single transaction: the event is de-duplicated by its primary key; a succeeded payment for a known attempt settles the subscription and issues the credential, guarded against out-of-order delivery; and an event for an unknown payment is **retained** for later reconciliation rather than dropped. Version 2 fixed a real gap here (see the bugs section) so that a retained event can actually be recovered, and added a sweep that re-attempts retained events whose payment later becomes known. [**PROVEN**] for signature verification on the deployed endpoint (valid -> 200, invalid -> 401); the settlement-from-a-real-delivery path is discussed honestly in section 12.

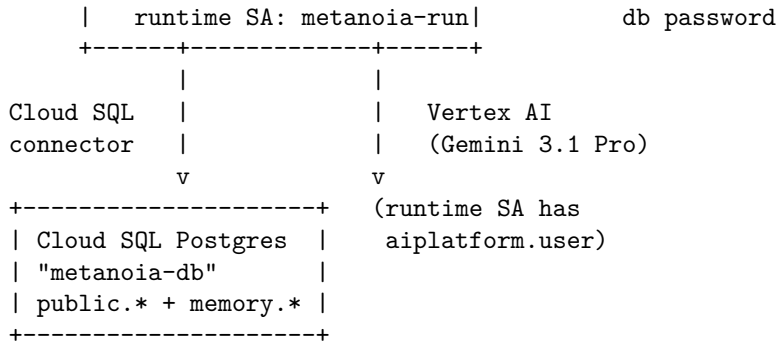
---

9. Deployment architecture: Cloud Run, Vertex AI, Secret Manager, and service accounts

Plain English: the app is live on Google Cloud. It runs as a container on Cloud Run, stores durable data in Cloud SQL, runs the Gemini agent on Vertex AI, and keeps its secrets in Secret Manager. It authenticates to all of these using a dedicated, least-privilege service account rather than any hard-coded keys.

Here is the deployment as a text diagram.





In words: a browser (or Hyperswitch) reaches the public Cloud Run service. The container is a lean Next.js standalone build. It runs as the service account `metanoia-run`, which has exactly three grants: connect to Cloud SQL, use Vertex AI, and read the specific secrets. The three secrets — the Hyperswitch secret key, the webhook signing key, and the database password — live in Secret Manager and are mounted into the service at runtime; none is baked into the image or committed to source. The database is the smallest Postgres tier, and the schema migrations were applied to it. The Gemini agent runs on Vertex AI using the service account's identity, which was confirmed on the live deployment (the agent route returned success on Cloud Run with the model actually invoked).

Two operational details worth stating: the container image is built by Cloud Build from a Dockerfile and pushed to Artifact Registry; and there is a teardown script that removes every resource, because Cloud SQL is the only meaningful ongoing cost. The publishable, non-secret values (like the browser publishable key) are provided at build time; the true secrets are provided only at runtime from Secret Manager. [**PROVEN**] — the deployed app serves its pages, completes a real checkout, reads and writes Cloud SQL, and runs the agent on Vertex.

---

10. Every important bug found, and how it was corrected

Plain English: this section is the engineering diary. Each entry is a real defect and the fix.

1. **Renewal could lose a payment on a crash.** The renewal charged first and recorded afterward, so a crash between the charge and the record would lose the payment. **Fix:** pending-first — write a durable pending attempt *before* the charge, using the same idempotency key, so a crash leaves an auditable row rather than a lost charge. [**CODED-UNTESTED**] against a real crash; the logic is in place and tested for idempotency.
2. **The capability probe returned 401 after a store change.** A storage refactor briefly broke cross-context visibility, so a credential issued while rendering the receipt was not visible to the provider route. **Fix:** restore disk-backed persistence that re-reads on each lookup. Now a valid credential returns 200. [**PROVEN**]
3. **"Denied" was shown when the honest answer was "no match."** A "ten-dollar transcription" request looked refused when the real reason was the catalog did not carry that category yet. **Fix:** add the transcription category and separate a true mandate refusal (Denied) from "nothing to compare" (No-match). [**PROVEN**]
4. **Resubscribing after a cancel did not charge.** Payment IDs were deterministic per customer, plan, and billing period, so cancelling and resubscribing in the same month reused the old succeeded payment and never charged again. **Fix:** detect an active subscription *directly*; reuse a payment ID only for retries of the same attempt (keyed by how many prior paid attempts exist for that plan); resubscribe after cancel now mints a fresh ID and charges again. [**PROVEN**] by a dedicated test.
5. **Retained webhook events could not be recovered.** An event for an unknown payment was retained but, on a later redelivery, was rejected as a duplicate before it could be reprocessed. **Fix:** only a fully processed event is a true duplicate; a retained event is reprocessed on redelivery once its

payment is known, plus a reconciliation sweep. [**PROVEN**] by two new tests.

6. **In-flight renewal statuses were marked as failed.** The renewal marked every non-succeeded status as failed, including "processing" and "requires customer action." **Fix:** only terminal statuses are marked failed; in-flight ones stay pending for a webhook or a later retrieve to settle. [**PROVEN**] (type/logic).
7. **A fabricated card number appeared on the receipt.** The receipt showed a hard-coded "VISA 4242." **Fix:** remove it; real masked details would come from the authoritative payment response. [**PROVEN**] (removed).
8. **A code comment overstated idempotency headers.** A comment claimed Hyperswitch has no idempotency header; the API reference shows one is documented by example but not on the current create-payment reference. **Fix:** treat the header as unverified for this endpoint and keep the stable merchant payment ID as the proven guard. [**CODED-UNTESTED**] for the header itself.
9. **A regression I introduced: the subscription cap.** While making the mandate editable, the maximum active-subscription bound was raised, which broke a bounds test. **Fix:** restore the intended maximum so the test passes again. [**PROVEN**] (tests green).
10. **Consent was overstated as "done."** An earlier note implied recurring consent was complete. In reality the SDK only captures consent when its saved-method control is enabled and ticked. **Fix:** enable that control, add a visible disclosure, and describe it honestly as an affordance, not a proven recurring setup. [**CODED-UNTESTED**]
11. **A misleading receipt label.** The receipt timeline hard-coded "webhook pending deployment," which stayed stale after deploying. **Fix:** relabel to an honest "webhook receiver verified; awaiting delivery." [**PROVEN**] (copy fixed).

---

11. Current test and verification status (actual repository numbers)

Plain English: the automated tests run on the fast in-memory backend and check the parts that must never break. These are the real current numbers from this repository.

- `npm test` -> **82 passing tests across 16 test files.** [**PROVEN**]
- `npx tsc --noEmit` -> clean (no type errors). [**PROVEN**]
- `npm run lint` -> clean (no warnings). [**PROVEN**]

What the suites cover: the spending gate (per-charge and monthly caps enforced, refusal deterministic); the ranking math (three choices, compliant pick, hard-failure flags, over-cap refusal); the propose-versus-decide boundary (server uses server prices, cannot be tricked into an over-cap plan, agent imports no payment functions); checkout (refuses over-cap without calling Hyperswitch, idempotent, ignores stale events, and the new resubscribe-after-cancel behavior); renewal (no double-count, refuses on a raised price, 409 without a saved method, idempotent); the scouts (output sanitized to shortlisted plans, lens scoping correct); preference memory (nothing stored without consent, deterministic profile synthesis, forget-all wipes state); webhook reconciliation (retained events recover on redelivery and via the sweep, and preserve their metadata); refunds (ownership refused with 403, non-succeeded refused with 409, idempotent duplicate, authoritative status from retrieval); session ownership (owner allowed, cross-session and tampered and anonymous all blocked); and the editable-mandate bounds.

Beyond unit tests, the following were observed live on the deployed app: the pages load; a real in-browser checkout completed and settled; the database-backed pages read Cloud SQL; the Gemini agent ran on Vertex from Cloud Run; the webhook endpoint verified a correctly signed payload (200) and rejected bad and missing signatures (401); and one real refund succeeded against the sandbox. [**PROVEN**]

---

12. What Changed Since Version 1

Version 1 described a working local prototype at 40 passing tests, explicitly not deployed, with the webhook and Cloud SQL paths coded but unobserved. Here is what is different now.

- **It is deployed.** The app runs live on Cloud Run with Cloud SQL, the Gemini agent runs on Vertex from the cloud, and secrets are in Secret Manager. Version 1 listed deployment as the next milestone; it is done.
- **Per-browser isolation was added.** The old single shared demo customer was replaced by an opaque per-session identity, so each browser is its own tenant and cannot see or refund another session's payments.
- **A Payment Test Lab was added,** with copy-only test cards for success, decline, insufficient funds, and 3DS, per-row live status, and a real server-side refund.
- **Refunds, cancellation, and resubscription** were added and made correct and idempotent, including the fix that resubscribing after a cancel charges again.
- **Recurring consent** became a visible affordance in checkout.
- **The webhook story was fully diagnosed** (section 13), including a neutral-collector control test.
- **Eleven bugs were fixed** (section 10), several of them real payment-correctness issues.
- **The marketplace expanded from 18 to 30 offers across 10 capabilities.** Four new categories were added — LLM inference, transactional email, observability, and authentication — each with three fictional vendors (budget, balanced, premium). Counts are now derived from the catalog, never hardcoded, and the same deterministic ranking, SpendGuard, and refinement work for every category.
- **The result screen gained two things that make the architecture legible.** A **Decision Authority** panel shows who decided what: the pipeline (Gemini extracted requirements, four advisory scouts, server ranking, SpendGuard, deterministic final selection), **MODEL PROPOSED vs SERVER FINAL**, a **SERVER OVERRIDE** with the exact reason when the server changes or rejects the model's pick, and the real ranking score parts. And the offers are split into **PURCHASABLE SANDBOX OFFERS** (fictional, onboarded, ranked, checkout-eligible) versus research-only **REAL-MARKET REFERENCES** (real companies from the Market scout, labeled "RESEARCH ONLY, NOT PURCHASABLE", never selectable, marked "not verified" when no source backs them).
- **Tests grew from 40 to 82** across 16 files, adding webhook reconciliation, refunds, session ownership, mandate bounds, the catalog expansion (category recognition, ranking, refusal, refinement), and the Decision Authority override rendering.
- **A payments audit document** was produced, grounded in official Hyperswitch documentation, correcting several earlier overconfident claims (for example, that a webhook idempotency header was documented, and that recurring consent was already complete).

Two honesty corrections also happened since Version 1: the Stripe recurring path was found to be blocked by a Stripe account capability (section 13), and the webhook delivery gap was traced carefully instead of being blamed on any one party.

(Note on the PRD: there is no `PRD.md` inside the app repository (`metanoia/`), but the original product requirements document does exist one level up in the workspace at `JusPay/PRD.md`. It frames the thesis ("agents are becoming the customer; this is the settlement layer for when an agent actually buys the subscription, on Hyperswitch") and the P0/P1 scope; this guide's product boundary is consistent with it and with `STATUS.md`.)

---

13. What Is Proven vs Deferred (and the exact limitations)

Plain English: this is the honest ledger. Be precise here — it is the section an interviewer will trust most.

13.1 Proven

- The propose-versus-decide safety model, the deterministic ranker, and SpendGuard.
- The agent cannot reach any payment function (test-enforced).

- A real Fauxpay customer-initiated checkout settles, both locally and on the live deployment.
- Stable payment IDs and idempotency (confirming twice records one subscription).
- Credential-gated capability proof: a valid credential returns 200, a bad or missing one returns 401.
- Per-browser session isolation and cross-session refusal.
- One real refund against the sandbox, plus the refund guardrail logic.
- Cancellation frees budget and revokes the credential; resubscribe-after-cancel charges again.
- The webhook **receiver**: it accepts valid signatures and rejects invalid ones on the deployed endpoint.
- Deployment: Cloud Run + Cloud SQL + Vertex + Secret Manager, verified working.
- 82 passing tests, clean type-check, clean lint.

13.2 The exact limitations, stated precisely

- **Fauxpay proves sandbox checkout, but not saved-card recurring (merchant-initiated) billing.** Fauxpay returns no reusable payment method.
- **Stripe merchant-initiated recurring remains blocked by Stripe raw-card API access.** Routing a customer-initiated payment to the Stripe connector returns an error because Hyperswitch forwards the card to Stripe and the connector needs raw-card API access enabled on the Stripe account, which is a Stripe-side request. So routing to Stripe was demonstrated, but a Stripe authorization was not.
- **The renewal scheduler is deferred.** There is no background job that charges subscriptions when they are due; renewal is a manual, on-demand action. Automatic recurring is therefore not demonstrated.
- **The webhook receiver accepts valid signatures and rejects invalid ones.** That much is proven on the deployed endpoint.
- **Hyperswitch successfully delivered a webhook to a neutral third-party collector.** In a control test, Hyperswitch's sandbox sent a signed POST to an independent collector, which received it within seconds. So Hyperswitch is capable of sending outbound webhooks.
- **Hyperswitch delivery to the Cloud Run endpoint was not observed.** Despite the receiver being verified and the endpoint reachable by ordinary clients, no Hyperswitch-originated request ever appeared in the Cloud Run request logs; Hyperswitch's own dashboard recorded the attempts as failed.
- **The exact reason the Cloud Run delivery did not land remains unknown.** This guide does not claim any specific cause. It does not assert that TLS, IPv6, HTTP/2, Hyperswitch, or Cloud Run is responsible; no single mechanism was confirmed. What was ruled out on our side was checked directly: the app code, cold start, the URL form, public-access permission, and ingress were all verified correct, and ordinary clients reach the endpoint normally.
- **Full end-to-end webhook delivery is not claimed, and real recurring payments are not claimed.** The receiver is proven; a real Hyperswitch-to-Metanoia delivery landing and settling is not.

13.3 Deferred (roadmap)

Automatic renewal scheduling; real off-session recurring once the Stripe capability is granted or a self-hosted/production Hyperswitch is used; smart routing, a second connector, and decline recovery; cryptographic AP2 mandate signatures; an x402 pay-per-call handshake for open discovery; and OAuth profile import with repository code analysis.

---

14. Glossary and interview demo script

14.1 Glossary (plain definitions)

- **Hyperswitch** — Juspay's open-source payment orchestrator: one API in front of many payment processors, with routing, connectors, and webhooks.
- **Connector** — a specific processor behind Hyperswitch. Here, Fauxpay (a dummy sandbox connector for checkout) and Stripe (reserved for the recurring path).

- **CIT, customer-initiated transaction** — a payment the customer is present for and confirms; the checkout.
- **MIT, merchant-initiated transaction** — a later, off-session charge triggered by the merchant or agent against a saved method; a renewal.
- **Mandate** — the user's standing authorization to spend under rules. Here it is a standing instruction plus a policy of caps that the spending gate enforces.
- **SpendGuard** — Metanoia's deterministic spending gate: the "server decides" authority.
- **Idempotency** — the property that retrying the same operation does not duplicate its effect; here enforced by stable, merchant-supplied payment IDs and database primary keys.
- **Webhook** — an asynchronous, signed HTTP callback from Hyperswitch reporting a payment's status; verified over the raw body with an HMAC signature.
- **Webhook receiver** — the endpoint that receives and verifies those callbacks. Distinguish it from "delivery," which is Hyperswitch actually sending the callback.
- **HMAC** — a keyed hash used to sign and verify a message; here it proves a webhook really came from Hyperswitch and was not altered.
- **3DS** — a step-up authentication challenge; in Hyperswitch it shows as a "requires customer action" status with a redirect.
- **Refund** — returning a settled payment; here it is server-side, owned by the session, idempotent, and verified by retrieval.
- **Session isolation** — scoping every user's payments and data to their own browser identity so no one can access another's.
- **Cloud Run** — Google's serverless container platform; where the app is deployed.
- **Cloud SQL** — Google's managed Postgres; the durable database.
- **Secret Manager** — Google's managed secret store; where the API keys and database password live.
- **Service account** — a non-human cloud identity the app runs as, granted only the permissions it needs.
- **Vertex AI / Gemini** — Google Cloud's model platform; the agent runs Gemini 3.1 Pro, the scouts run a faster Gemini model.
- **Drizzle** — the typed schema-and-query layer used with Postgres.
- **AP2** — an emerging standard for agent-carried payment mandates; modeled here as shapes, with cryptographic signatures deferred.
- **x402** — an HTTP-402-based pay-per-call settlement pattern for open agent commerce; deferred here.
- **Grounding** — constraining a model to verifiable sources; the market scout uses Google Search so its external claims are backed by real results.

14.2 Step-by-step interview demo script (about five minutes)

1. **Frame it (0:00).** "Metanoia gives an agent a budget, not your card. The model proposes; the server decides; Hyperswitch settles." Point at the three mandate cards.
2. **Run a real request (0:40).** Click the transcription preset, press Run. Show the honest processing screen, then the three ranked offers, the agent trace, the spending audit in order, and the four scout lenses. Note which plan wins and which is blocked for being over budget.
3. **Show the refusal (1:40).** Click the over-budget preset, Run, and show the red Denied screen with the exact failing check and "card never touched."
4. **Show the honest no-match (2:10).** Type a capability the catalog does not carry, Run, and show the neutral "nothing to compare" screen. Explain the fictional-vendor boundary.
5. **Buy it (2:40).** Re-run transcription, confirm the winner, and pay with the Visa test card inside Hyperswitch's secure form. Point out the recurring-consent control and that the card never touches your server.
6. **Prove the capability (3:30).** On the receipt, walk through the payment-processing timeline and the authenticated sandbox provider returning 200. Stress that it is a sandbox provider, not an external vendor.
7. **Show payment depth (4:10).** Open the Payment Test Lab. Copy a decline card and show a real decline, then run a real refund on a succeeded payment and show the verified status. Mention session

isolation: this list is only your browser's payments.

8. **Close on architecture and honesty (4:40).** One agent proposes; a deterministic ranker and SpendGuard decide; four advisory scouts; Hyperswitch settles; it is deployed on Cloud Run with Cloud SQL and Vertex. Then state plainly what is proven and what is not: checkout and refunds are real; recurring is blocked by a Stripe capability; and the webhook receiver is verified while a real Hyperswitch-to-Cloud-Run delivery was not observed and its cause is still unknown.

---

15. Strongest architectural decisions, honest weaknesses, and likely interviewer questions

15.1 The strongest decisions

- **The model proposes, the server decides.** A deterministic gate on server-owned data is testable, explainable, and cannot be prompt-injected into overspending. This is the decision the whole product rests on, and it is enforced by four independent barriers plus a test.
- **Honesty as a feature.** Fictional vendors labeled as such; a separate "no match" state; an authenticated sandbox provider labeled as a sandbox; and a webhook story that was *diagnosed* with a neutral-collector control test instead of hand-waved. This honesty is more convincing to a payments team than a green checkmark that cannot be explained.
- **Correctness pushed into the database.** Idempotency and webhook de-duplication are primary-key constraints, not application checks, so they hold under concurrency.
- **Least-privilege, secret-clean deployment.** A dedicated service account, secrets only in Secret Manager, nothing baked into the image or committed.

15.2 Honest weaknesses

- Real recurring (merchant-initiated) billing is not proven; it is blocked by a Stripe account capability, and the automatic scheduler is deferred.
- A real Hyperswitch-to-Metanoia webhook delivery was never observed on Cloud Run, and the exact cause is unknown.
- The vendors are fictional by design, so the demo proves the mechanism, not a third-party vendor integration.
- The capability proof calls an internal sandbox provider, not a real outside API.

15.3 Likely interviewer questions, with honest answers

- **How do you stop the agent from overspending?** A deterministic spending gate evaluates the mandate on server-owned prices before any charge; the agent has no payment tool; and the checkout route re-checks and returns 403 before Hyperswitch is called.
- **Is this real recurring billing?** No. Fauxpay proves the first payment; merchant-initiated recurring needs a Stripe capability that is not granted, and the scheduler is deferred.
- **Did the webhook work end to end?** The receiver is proven — it accepts valid signatures and rejects invalid ones. Hyperswitch delivered to a neutral collector, but delivery to Cloud Run was not observed, and the exact cause is unknown; no specific mechanism was confirmed.
- **How is a refund safe on a public app?** It is server-side, must be owned by the requesting session, must be for a succeeded payment, is idempotent by a deterministic refund ID, and reports the authoritative status from retrieval.
- **How do two users not see each other's payments?** Each browser has an opaque, unguessable session identity that scopes all payment data; ownership is an exact match, and tampered or cross-session identities own nothing (tested).
- **Where does AI touch money?** Nowhere directly. It only produces a structured proposal; authorization and settlement are deterministic and server-side.

- **What breaks at scale?** In-memory state would be per-instance on serverless, which is why the durable path is Cloud SQL Postgres with idempotency and de-duplication as database constraints — and that path is now deployed.

Generated from `docs/METANOIA-WALKTHROUGH-V2.md`*. Code snapshot: commit* `4dfc45b`*. No credentials, secrets, database passwords, or full API keys are included in this document.*